

Rešavanje problema raspoređivanja poslova po mašinama metaheuristikama

Minimum Job Shop Scheduling

Nenad Pešić

septembar 2026.

Sadržaj

1	Uvod	2
2	Opis rešenja	3
3	Eksperimentalni rezultati	4
3.1	Okruženje za eksperimentisanje	4
3.2	Test instance	4
3.3	Metodologija	5
4	Zaključak	7
5	Literatura	8

Poglavlje 1

Uvod

Poglavlje 2

Opis rešenja

Poglavlje 3

Eksperimentalni rezultati

3.1 Okruženje za eksperimentisanje

Kako bismo olakšali kasniju interpretaciju rezultata, za početak ćemo dostaviti kontekstualne informacije o veličini eksperimenata i okruženju na kojima su eksperimenti pokretani.

Sve eksperimentalne skripte izvršavane su na privatnom računar, čije se najznačajnije karakteristike nalaze u sledećoj tabeli:

Tabela 3.1: Karakteristike računara na kome su izvršena sva merenja.

komponenta	vrednost
procesor	Intel Core Ultra 7 165U
frekvencija	3,71 GHz
broj jezgara	12 fizičkih / 14 logičkih
radna memorija	62,2 GB
operativni sistem	Ubuntu 24.04.4 LTS, 7.0.0-28-generic kernel, x86-64
standardna biblioteka	glibc 2.39
programski jezik	Python 3.12.3, GCC 13.3.0
egzaktni rešavač	IBM ILOG CPLEX 22.2.0.0
biblioteka za modelovanje	PuLP 3.3.2

Vredi napomenuti da su sve razvijene metaheuristike implementirane tako da se izvršavaju na **jednoj niti** bez mogućnosti paralelizacije. Egzaktni rešavač (CPLEX) pokretan je sa podrazumevanim brojem niti, dakle sa svih 14 logičkih jezgara. Zbog te razlike, metode je najpreciznije porediti na osnovu broja poziva funkcije dekodiranja, a ne po utrošenom vremenu. Dužina izvršavanja navedena je kao dodatni, orijentacioni podatak.

3.2 Test instance

Pri evaluaciji razvijenih metoda korišćeno je devet različitih test instanci. Od njih devet, osam je preuzeto iz zbirke OR-Library (Beasley 1990). Instance `ft06`, `ft10` i `ft20` potiču iz rada (Fisher i Thompson 1963), dok instance `1a01` - `1a05` dolaze iz (Lawrence 1984). Poslednja, deveta, instanca `mini3` kreirana je za potrebe ovog rada. Predstavlja relativno malu instancu čiji se op-

timum može pronaći algoritmom grube sile. Kao takva služila je najpre pri validaciji ispravnosti implementiranih metoda optimizacije. Pregled najvažnijih informacija o svim instancama dat je u narednoj tabeli.

Tabela 3.2: Test instance sa dimenzijama, donjim granicama i objavljenim optimalnim vrednostima.

instanca	$n \times m$	operacija	donja granica	optimum	izvor optimuma
mini3	3×3	9	10	11	iscrpna pretraga
ft06	6×6	36	47	55	(Fisher i Thompson 1963)
la01	10×5	50	666	666	donja granica
la02	10×5	50	635	655	(Lawrence 1984)
la03	10×5	50	588	597	(Lawrence 1984)
la04	10×5	50	537	590	(Lawrence 1984)
la05	10×5	50	593	593	donja granica
ft10	10×10	100	655	930	(Fisher i Thompson 1963)
ft20	20×5	100	1119	1165	(Fisher i Thompson 1963)

Donja granica u tabeli 3.2 predstavlja veću od dve trivijalne granice: (1) najdužeg posla i (2) najopterećenije mašine. Način implementacije računa detaljno je opisan u odeljku o donjim granicama. Kod instanci **la01** i **la05** ta granica se poklapa sa optimumom, pa svako rešenje koje je dostigne ujedno i **dokazuje** svoju optimalnost, bez potrebe za egzaktnim rešavačem.

Instance su izabrane tako da pokriju što širi opseg težine. Konkretno, **mini3** i **ft06** sve razvijene metode rešavaju do optimuma, pa služe kao provera ispravnosti. Instance **la01**–**la05** razdvajaju metode po pouzdanosti. **ft10** i **ft20** su, uprkos skromnim dimenzijama, poznato teške. Instanca **ft10** je ostala nerešena punih dvadeset šest godina nakon objavljivanja (Jain i Meeran 1999).

3.3 Metodologija

Za početak je potrebno objasniti način izbora veličine eksperimenta. Bilo je potrebno održati balans između validnosti rezultata i razumne upotrebe (pre svega vremenskih) resursa. Sa jedne strane, ne možemo dopustiti da eksperimente izvršavamo na premalom uzorku i time dovedemo u pitanje opštost zaključaka, dok je sa druge strane potrebno da se eksperimenti izvrše u razumnom vremenskom roku.

Kao što je ranije navedeno, zbog nedostatka konkurentnosti u implementacijama algoritama, efikasnost različitih optimizacionih metoda nije pogodno upoređivati sa egzaktnim rešavačem na osnovu vremena izvršavanja. Kod međusobnog poređenja situacija je nešto bolja, ali se pokazalo veoma izazovno unapred dozvoliti funkciji određeni vremenski okvir, pa je vreme izvršavanja zauzelo ulogu *retrospektivne mere*, dok su sami algoritmi unapred ograničeni brojem pozivanja funkcije dekodiranja.

Svakom algoritmu je unapred data gornja granica broja poziva ove funkcije i taj broj nazivamo *budžetom* funkcije. Ispostavlja se da je budžet od oko 200,000 poziva dekodera sasvim dovoljan

za verodostojno reprodukovanje rezultata, a opet dovoljno mali da poziv optimizacione metode ostane dovoljno vremenski ograničen. Primećujemo da se vreme izvršavanja optimizacionog metoda razlikuje i pri istom budžetu, a razlog tome nalazi se u specifičnostima samih algoritama, gde se vreme “rasipa” na pozive drugih pomoćnih metoda.

— slike idu ovde

Poglavlje 4

Zaključak

Poglavlje 5

Literatura

- Beasley, J. E. 1990. „OR-Library: Distributing Test Problems by Electronic Mail“. *Journal of the Operational Research Society* 41 (11): 1069–72.
- Fisher, H., i G. L. Thompson. 1963. „Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules“. U *Industrial Scheduling*, uredio J. F. Muth i G. L. Thompson, 225–51. Englewood Cliffs, New Jersey: Prentice Hall.
- Jain, A. S., i S. Meeran. 1999. „Deterministic Job-Shop Scheduling: Past, Present and Future“. *European Journal of Operational Research* 113 (2): 390–434.
- Lawrence, S. 1984. „Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques (Supplement)“. Pittsburgh, Pennsylvania: Graduate School of Industrial Administration, Carnegie-Mellon University.